Resource Cleanup in C and the R API

Photo by Steve Buissinne

2019/05/22

r-lib, cleancall

Gábor Csárdi, Lionel Henry

Introduction

We have just released the cleancall package to CRAN. cleancall is a new package that helps avoiding resource leaks in the C code of your R package. In this blog post, we show some example resource leaks from base R and R packages, and also show three different ways to fix them, one of which uses the cleancall package.

The problem

When writing C code in R packages, one non-trivial issue is to make sure that resources acquired by a C function are correctly released or wrapped before giving back control to R. The difficulties partially arise from the many ways a C function can terminate and transfer control back to R:

- regular termination,
- error,
- warning or other condition caught with `tryCatch()`,
- debugger exit,
- keyboard interruption.

Resources that need to be released include:

- memory areas,
- file handles,
- connections (sockets),
- database connections,
- Windows handles for threads, events, etc.,
- others.

Regular termination

Most functions terminate successfully, most of the time. Releasing resources is easy in this case, we can simply do it at the end of the function, before returning.

Error

Errors may happen two ways. They can be thrown directly, or by the R API. A function that reads a file will throw an error it if fails to open the file for reading. These errors are easy to handle, we can release all resources before throwing the error.

Errors can also be thrown by the R API. The R C API functions do not return on error, instead they perform an error (a `longjmp` technically), that can be caught by R or (recently) C code. Releasing resources is trickier in this case, as our C function never gets back the control after the failed R API call.

Warning or other condition caught with `tryCatch()`

Usually we assume that throwing a warning (or other non-error condition) does not cause an early exit, but this is not always the case. Warnings can be caught by an exiting `tryCatch()`.

Debugger exit

If the C function calls back to R to evaluate R expressions, then these might invoke the debugger, see `?debug` or `?trace`. The `Q` debugger command invokes the `abort` restart, to get back to the top-level prompt. In this case most functions on the (R and C) stack are removed at once.

Keyboard interruption

C code can call `R_CheckUserInterrupt()`, and it is indeed suggested to do so during long computation or I/O. If the user initiates an interrupt via the keyboard or a signal, then `R_CheckUserInterrupt()` never returns, and the control goes back to top level.

To illustrate the resource cleanup problem more, we'll show some examples from base R and CRAN packages that contain potential resource leaks.

Example 1: `download.file(method = "internal")`

The C implementation of `download.file()` opens a file for saving the downloaded data to, and it does not clean or even close this file on a keyboard interrupt. The file is opened here: https://github.com/wch/r-source/blob/f3a67c36c5cf4c715dac374e33168cfc348106aa/src/modules/internet/internet.c#L553

```
    FILE *out;
    [...]
    out = R_fopen(R_ExpandFileName(file), mode);
    [...]
```

We can easily show the problem in practice as well. First, we create a handy function that interrupts the current process one second after calling it. This allows us to simulate an interrupt from the keyboard. It starts a background process that will send an interrupt (SIGINT on Unix, CTRL+C on Windows) to our R process.

```
interrupt <- function(expr, after = 1) {
  tryCatch({
    p <- callr::r_bg(function(pid, after) {
      Sys.sleep(after)
      ps::ps_interrupt(ps::ps_handle(pid))
    }, list(pid = Sys.getpid(), after = after))
    expr
    p$kill()
  }, interrupt = function(e) e)
}
```

`ps::ps_open_files()` lists all open files of a process, `ps::ps_handle()` returns a handle for the current R process. You should see the three file descriptors corresponding to standard input, output and error here, and another file is used by the blogdown engine rendering this page. If you run this code in your R session you'll not see that file, but of course you may see others.

```
ps::ps_open_files(ps::ps_handle())
#> # A tibble: 4 x 2
#>      fd path
#>   <int> <chr>
#> 1     0 /dev/ttys021
#> 2     1 /dev/ttys021
#> 3     2 /dev/ttys021
#> 4     7
/Users/gaborcsardi/R/blogdown/scripts/render_page.R
```

Now we will start downloading a file, and while it is downloading, we'll send an interrupt to our R process, causing an early exit.

```
interrupt(
  download.file("http://httpbin.org/delay/3", tempfile(),
method = "internal")
)
#> <interrupt: >
```

The list of open files will now include the one opened by `download.file()`:

```
ps::ps_open_files(ps::ps_handle())
#> # A tibble: 5 x 2
#>      fd path
#>   <int> <chr>
#> 1     0 /dev/ttys021
#> 2     1 /dev/ttys021
#> 3     2 /dev/ttys021
#> 4     7
/Users/gaborcsardi/R/blogdown/scripts/render_page.R
#> 5     9
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…
```

Because `download.file()` fails to clean up this file handle, it is not released until the R session quits. On Windows this file is probably locked, and cannot be removed until R exits.

Example 2: `download.file(method = "libcurl")`

There is a very similar bug in the implementation of the `download.file()` libcurl method, which opens an output file here: https://github.com/wch/r-source/blob/def075db88ae87104c38437a57e9327b078bb804/src/modules/internet/libcurl.c#L566 but it does not close it on error or interrupt. This method can download many files in parallel, in which case multiple file handles may be lost:

```
ps::ps_open_files(ps::ps_handle())
#> # A tibble: 5 x 2
#>      fd path
#>   <int> <chr>
#> 1     0 /dev/ttys021
#> 2     1 /dev/ttys021
#> 3     2 /dev/ttys021
#> 4     7
/Users/gaborcsardi/R/blogdown/scripts/render_page.R
#> 5     9
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

interrupt(
  download.file(
    rep("https://httpbin.org/delay/3", 3),
paste0(tempfile(), 1:3),
    method = "libcurl")
)
#> <interrupt: >
ps::ps_open_files(ps::ps_handle())
#> # A tibble: 8 x 2
```

```
#>      fd path
#>   <int> <chr>
#> 1     0 /dev/ttys021
#> 2     1 /dev/ttys021
#> 3     2 /dev/ttys021
#> 4     7
/Users/gaborcsardi/R/blogdown/scripts/render_page.R
#> 5     9
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 6    15
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 7    16
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 8    17
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…
```

Example 3: `edit()`

`edit()` invokes the text editor specified by the `editor` option, with the supplied object to be edited. `edit()` can also write the edited version to a file. It opens a file here to create its first version, before starting the editor: https://github.com/wch/r-source/blob/019f16d3dec4f97c1b4e4f0ec905148e36979e37/src/main/edit.c#L121-L128

```
121    if((fp=R_fopen(R_ExpandFileName(filename), "w"))
== NULL)
122        errorcall(call, _("unable to open file"));
123    if (LENGTH(STRING_ELT(fn, 0)) == 0) EdFileUsed++;
124    PROTECT(src = deparse1(x, 0, FORSOURCING)); /*
deparse for sourcing, not for display */
125    for (i = 0; i < LENGTH(src); i++)
126        fprintf(fp, "%s\n",
translateChar(STRING_ELT(src, i)));
127    UNPROTECT(1); /* src */
128    fclose(fp);
```

If all goes well, then it will close it just 7 code lines later. However, it also calls the R API a number of times before closing the file, so if any of these API calls fail, the file is never closed. In particular, it calls `deparse1()` which is roughly equivalent to the R `deparse()` function, and `deparse()` fails on long vectors:

```
ps::ps_open_files(ps::ps_handle())
#> # A tibble: 8 x 2
#>      fd path
#>   <int> <chr>
#> 1     0 /dev/ttys021
#> 2     1 /dev/ttys021
#> 3     2 /dev/ttys021
#> 4     7
/Users/gaborcsardi/R/blogdown/scripts/render_page.R
#> 5     9
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 6    15
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 7    16
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 8    17
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

edit(1:10e10, file = tempfile())
#> Error in edit.default(1:1e+11, file = tempfile()):
long vectors not supported yet: ../../../../R-
3.5.3/src/include/Rinlinedfuns.h:519
ps::ps_open_files(ps::ps_handle())
#> # A tibble: 9 x 2
#>      fd path
#>   <int> <chr>
#> 1     0 /dev/ttys021
#> 2     1 /dev/ttys021
#> 3     2 /dev/ttys021
#> 4     7
/Users/gaborcsardi/R/blogdown/scripts/render_page.R
#> 5     8
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…
```

```
#> 6     9
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 7    15
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 8    16
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…

#> 9    17
/private/var/folders/59/0gkmw1yj2w7bf2dfc3jznv5w0000gn/T/Rtmp0Uww3…
```

Notice that we have one more extra open file here.

Example 4: the wait() method in processx:

`processx::process` is an R6 class for a subprocess. Its `$wait()` method waits for the subprocess to finish, with a timeout. To implement `$wait()` on Unix, processx opens a pair of pipe file descriptors. These are temporary and should be closed once the function exits. `$wait()` is interruptible, it calls `R_CheckUserInterrupt()` periodically. However, in the current, 3.3.0 version of processx it does not close the pipe file descriptors on an interrupt. Here is an illustration:

```
ps::ps_num_fds(ps::ps_handle())
#> [1] 20

p <- processx::process$new("sleep", "10")
interrupt(
  p$wait()
)
#> <interrupt: >
p$kill()
#> [1] TRUE
```

```
gc()
#>           used (Mb) gc trigger (Mb) limit (Mb) max
used (Mb)
#> Ncells  626041 33.5    1203691 64.3         NA
1203691 64.3
#> Vcells 1188386  9.1    8388608 64.0      16384
2160124 16.5

ps::ps_num_fds(ps::ps_handle())
#> [1] 22
```

`ps_num_fds()` prints the number of open file descriptors of a process. The two extra file descriptors that are open after garbage collection are the two ends of the (supposedly) temporary pipe, just opened by `$wait()`.

Fix 1: External pointer and finalizer

One generic solution to resource cleanup is to wrap all C resources into one or more R external pointer objects, and add finalizers to them. We show how this can fix the processx problem in the previous example.

This is how the `processx_wait()` C function looks like before the fix: https://github.com/r-lib/processx/blob/a8f09d147fead78347a87fcf4e0fbd1c07de1c21/src/unix/processx.c#L507-L589

First, we need to create a finalizer function, that will be called by the R garbage collector, after `$wait()` has finished, at the next garbage collection:

```
static void processx__wait_finalizer(SEXP xptr) {
  SEXP tag = R_ExternalPtrTag(xptr);
  if (INTEGER(tag)[0] >= 0) close(INTEGER(tag)[0]);
```

```
  if (INTEGER(tag)[1] >= 0) close(INTEGER(tag)[1]);
}
```

An external pointer can have a *tag*, which is an R object that is kept alive as long the pointer object itself is alive. In this case we can put the file descriptors in the tag, in an integer vector of length two. In more complicated cases the resources cannot easily be represented as R objects, so you would use the actual C pointer, with a custom C `struct` to store them.

Now we need to create the external pointer, before the pipes are opened:

```
  SEXP tag = PROTECT(allocVector(INTSXP, 2));
  INTEGER(tag)[0] = INTEGER(tag)[1] = -1;
  SEXP xptr = PROTECT(R_MakeExternalPtr(NULL, tag,
R_NilValue));
  R_RegisterCFinalizerEx(xptr, processx__wait_finalizer,
0);
```

We initialize the file descriptors to -1, which is guard value, meaning that no cleanup is needed.

We can now open the pipes and save their file descriptors in the tag of the external pointer.

```
  if (pipe(handle->waitpipe)) {
    processx__unblock_sigchld();
    error("processx error: %s", strerror(errno));
  }
  INTEGER(tag)[0] = handle->waitpipe[0];
  INTEGER(tag)[1] = handle->waitpipe[1];
```

At the end of the function, we need to unprotect the external pointer, and the tag:

```
  UNPROTECT(2);
  return ScalarLogical(ret != 0);
}
```

One potential problem with the external pointer fix is that the resources will only be cleaned up at the next garbage collection, and sometimes this is too late. For example, if an open file is locked by the operating system, then we won't be able to remove that file, or its directory, until the garbage collector runs and closes it. It is easier to program if resource cleanup is immediate, and luckily the other two fixes below are.

Fix 2: the `R_ExecWithCleanup()` function

`R_ExecWithCleanup()` is a function in the R API, that can add a cleanup function to a regular C function call. The cleanup function is always executed, even on early exit:

```
SEXP R_ExecWithCleanup(SEXP (*fun)(void *), void *data,
              void (*cleanfun)(void *), void
*cleandata);
```

`R_ExecWithCleanup()` calls `fun` with `data`, and then calls `cleanfun` with `cleandata`. If `fun` calls the R API and exits early, then it will still call `cleanfun` with `cleandata`, before throwing the error up the stack.

To fix `$wait()` with `R_ExecWithCleanup()`, we'll need to define a cleanup function first. The cleanup function receives the cleanup data as a `void` pointer, we'll define a `struct` for this. For simplicity, this struct will also include the arguments to `processx_wait()`, so we can use the same struct for both `fun` and `cleanupfun`.

```
struct processx_wait_data {
  SEXP status;
  SEXP timeout;
```

```
  int fds[2];
};

void processx_wait_cleanup(void *data) {
  struct processx_wait_data *pdata = data;
  if (pdata->fds[0] >= 0) close(pdata->fds[0]);
  if (pdata->fds[1] >= 0) close(pdata->fds[1]);
}
```

The new `processx_wait()` function will call `R_ExecWithCleanup()`, to call the original `processx_wait()`, under a new name:

```
SEXP processx_wait_internal(void *data);
SEXP processx_wait(SEXP status, SEXP timeout) {
  struct processx_wait_data pdata = { status, timeout, {
-1, -1 } };
  SEXP result = R_ExecWithCleanup(processx_wait_internal,
&pdata,
                                  processx_wait_cleanup,
&pdata);
  return result;
}
```

`processx_wait_internal()` is very much like `processx_wait()` used to be, but we need to extract the arguments from the struct at the beginning:

```
SEXP processx_wait_internal(void *data) {
  struct processx_wait_data *pdata = data;
  SEXP status = pdata->status;
  SEXP timeout = pdata->timeout;
  int *fds = pdata->fds;
  ...
```

Now the only other thing we need to do is saving the file descriptors in the data struct:

```
  if (pipe(handle->waitpipe)) {
    processx__unblock_sigchld();
    error("processx error: %s", strerror(errno));
```

```
  }
  fds[0] = handle->waitpipe[0];
  fds[1] = handle->waitpipe[1];
```

Fix 3: the cleancall package

`R_ExecWithCleanup()` is a good fix to the resource cleanup problem, but it can be verbose and error prone. It also requires that you replace your original function with a wrapper that packs the original function arguments into a `struct` and an internal function that only has a `void*` argument.

We created the cleancall package to make resource cleanup easier. This package automates wrapping your functions with `R_ExecWithCleanup()`. cleancall has been just published on CRAN. Here we show how to use it to fix the processx `$wait()` method.

To use it in your package, you need to specify cleancall as a dependency, both as `LinkingTo` and `Imports`:

```
...
Imports: cleancall
LinkingTo: cleancall
...
```

cleancall defines the `call_with_cleanup()` R function and the `r_call_on_exit()` and `r_call_on_early_exit()` C functions.

You need to replace `.Call()` with `call_with_cleanup()` in your R code:

```
cleancall::call_with_cleanup(c_processx_wait,
private$status,
```

```
                                  as.integer(timeout))
```

In your C code, include the `cleancall.h` header:

```
#include <cleancall.h>
```

Next, create a cleanup function for the resource type:

```
void processx__close_fd(void *ptr) {
  int *fd = ptr;
  if (*fd >= 0) close(*fd);
}
```

This cleanup function closes a file descriptor. Once your resource type has a cleanup function, you can call `r_call_on_exit()` or `r_call_on_early_exit()` every time you acquire a resource of that type. Use `r_call_on_exit()` if the resource must be released on normal termination as well, and `r_call_on_early_exit()` if it must be released on early termination only:

```
  if (pipe(handle->waitpipe)) {
    processx__unblock_sigchld();
    error("processx error: %s", strerror(errno));
  }
  r_call_on_exit(processx__close_fd, handle->waitpipe);
  r_call_on_exit(processx__close_fd, handle->waitpipe +
1);
```

Typically `r_call_on_exit()` is more convenient for temporary resources. `r_call_on_early_exit()` is more convenient if the C function returns a handle (e.g. external pointer), for which it needs to allocate resources gradually. If all resource allocations are successful, and the function returns normally, then no cleanup is needed. However if an intermediate step fails, you need to release the resources acquired before the failure.

In does not matter much in this simple example, but it in important

in general that exit handlers are always executed in reverse order (last one in is the first one out, LIFO). This makes it easy to build a resource gradually. Exit handlers installed via `r_call_on_exit()` and `r_call_on_early_exit()` share the same stack.

We suggest that exit handlers are kept as simple and fast as possible. In particular, errors (and other early exits) triggered from exit handlers are not caught currently. If an exit handler exits early the others do not run. If this is an issue, you can wrap the exit handler in `R_tryCatch()` (available for R 3.4.0 and later). But in general the best error handlers do not call the R API at all.

What about C++?

The resource cleanup problem is also present if you interface C++ code. While cleancall can be used with C++, it works best with C code, since it follows C idioms.

Using external pointers and finalizers works for C++ as well, as does wrapping R API calls in `R_ExecWithCleanup()`.

Alternatively, you can wrap your R API calls in `R_tryCatch()` (R 3.4.0 and later), or use `R_UnwindProtect()` (R 3.5.0 and later).

If your C++ code needs to support older R versions, that is more challenging, and one possibility is to call back to R and call `tryCatch()` there.

Summary

Resource cleanup in C code can be challenging, especially given that there is not very much documentation on this topic.

Hopefully this post and the cleancall package will make resource cleanup much simpler, and fewer R packages will suffer from resource leaks in the future.

The tidyverse is proudly supported by